Follow step by step to imitate session 2 code (be careful with spelling mistakes in code and line breaks, when copying terminal code)

Step 0: Open VS Code in the project folder

In VS Code terminal, first go to your project root:

Now activate the virtual environment by running below code in terminal:

.\.venv\Scripts\Activate

You should see:

```
(.venv) PS C:\Users\ADMIN\projects\healthcare_agent>
```

If you do not see `(.venv)`, stop here and fix that first.

Step 1: Quick sanity check before Session 2

Before we add new code, confirm Session 1 modules still work.

Run in terminal:

```
python -c "from core.diagnostics import SymptomDiagnosticEngine;
engine=SymptomDiagnosticEngine(); print(engine.diagnose('chest pain and
sweating')['top_diagnosis']['specialty'])"
```

Expected result should be something like:

```
(.venv) PS D:\healthcare ai agent\healthcare_agent> python -c "from core.diagnostics import SymptomDiagnosticEngine; engine=SymptomDiagnosticEngine(); print(engine.diagnose('c
hest pain and sweating')['top_diagnosis']['specialty'])"
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|                                                                                | 103/103 [00:00<00:00, 151.26it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Cardiology
```

Then run in terminal:

python -c "from core.insurance import InsuranceEstimator; est=InsuranceEstimator(); print(est.estimate('demo123', 35, 'male', 29.5, 1, 'no', 'southeast')[1])"

Expected result should look like:

```
(.venv) PS D:\healthcare ai agent\healthcare_agent> python -c "from core.insurance import InsuranceEstimator; est=InsuranceEstimator(); print(est.estimate('demo123', 35, 'male
', 29.5, 1, 'no', 'southeast')[1])"
Estimated insurance cost: 5,407 (no prior history).
```

If both work, your backend is ready for Session 2.

Step 2: Install Transformers stack

Since Session 1 already installed sentence-transformers, a lot may already be there, but let's make sure.

Add in your requirements.txt

transformers
sentencepiece
accelerate

Why these?

- `transformers` → to load FLAN-T5
- `sentencepiece` → required by T5 tokenizer
- `accelerate` → helps model loading and inference

and then run in terminal again:

pip install -r requirements.txt

Step 3: Create core\explainer.py

Run in terminal:

ni core\explainer.py

Now paste this code into the file.

You can paste directly into the VS Code editor. That is safer than terminal for a long file. Open core\explainer.py and paste following code there:

```
from typing import Dict, Any, Optional
import threading

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM


class AIExplainer:
    """
    Bootcamp-friendly explanation layer.

    Main design:
    1. Build a strong deterministic explanation first.
    2. Optionally use FLAN-T5-Large to rewrite that explanation in more natural
language.
    3. If model output is weak or fails, safely fall back to the deterministic
version.
    """

    def __init__(
        self,
        model_name: str = "google/flan-t5-large",
        use_local_model: bool = False,
    ):
        self.model_name = model_name
        self.use_local_model = use_local_model
        self.tokenizer = None
        self.model = None
        self.model_loaded = False
        self.load_error: Optional[str] = None
        self._lock = threading.Lock()

        if self.use_local_model:
            self._try_load_model()

    def _try_load_model(self):
        try:
            print(f"Loading explainer model: {self.model_name}")
            self.tokenizer = AutoTokenizer.from_pretrained(self.model_name)
            self.model = AutoModelForSeq2SeqLM.from_pretrained(self.model_name)
            self.model_loaded = True
            print("Explainer model loaded successfully.")
        except Exception as e:
```

```
            self.model_loaded = False
            self.load_error = str(e)
            print("Explainer model could not be loaded. Falling back to
deterministic explanation.")
            print(f"Reason: {self.load_error}")

    def enable_local_model(self):
        if not self.model_loaded:
            self.use_local_model = True
            self._try_load_model()

    def _safe_get_top(self, payload: Dict[str, Any]) -> Dict[str, Any]:
        return payload.get("top_diagnosis", {}) or {}

    def _build_structured_explanation(self, payload: Dict[str, Any]) -> str:
        top = self._safe_get_top(payload)
        doctors = payload.get("doctors", []) or []
        insurance_text = payload.get("insurance_text", "Not available.")
        history_summary = payload.get("history_summary", "")

        if not top:
            return (
                "The system could not generate a clear health insight from the
current input. "
                "Please review the symptoms entered and try again."
            )

        diagnosis = top.get("diagnosis", "Unknown")
        specialty = top.get("specialty", "Unknown")
        urgency = top.get("triage_level", "Unknown")
        tests = top.get("recommended_tests", "Not available")
        advice = top.get("advice", "Not available")

        lines = []
        lines.append("Here is a simple explanation of the result.")
        lines.append("")
        lines.append(
            f"Based on the symptoms entered, the system found a health insight
that is most closely related to: {diagnosis}."
        )
        lines.append(
            f"The recommended next step is to consult a {specialty} specialist."
        )
        lines.append(
            f"The urgency level for this case is marked as {urgency}."
```

```
        )
        lines.append(
            f"Suggested tests include: {tests}."
        )
        lines.append(
            f"General advice from the support system is: {advice}"
        )

        if insurance_text:
            lines.append("")
            lines.append(insurance_text)

        if history_summary:
            lines.append("")
            lines.append(f"History note: {history_summary}")

        if doctors:
            lines.append("")
            lines.append("Suggested doctors based on the selected city and
specialty include:")
            for doc in doctors[:3]:
                lines.append(
                    f"- {doc.get('name', 'Unknown')} in {doc.get('city',
'Unknown')} "
                    f"({doc.get('specialty', 'Unknown')}, rating
{doc.get('rating', 'N/A')})"
                )

        lines.append("")
        lines.append(
            "This explanation is for educational decision-support purposes only
and is not a medical diagnosis. "
            "Please consult a licensed healthcare professional."
        )

        return "\n".join(lines)

    def _build_rewrite_prompt(self, structured_text: str) -> str:
        prompt = f"""
Rewrite the following healthcare support explanation in simple, calm, patient-
friendly English.

Rules:
- Do not add new medical facts.
- Do not remove important information.
```

```
- Do not use the word diagnosis.
- Keep the meaning unchanged.
- Keep it clear and reassuring.
- Keep it between 5 and 8 sentences.
- End with: Please consult a licensed healthcare professional.

Text:
{structured_text}
"""
        return prompt.strip()

    def _generate_with_flan(self, prompt: str) -> str:
        if not self.model_loaded or self.model is None or self.tokenizer is None:
            raise RuntimeError("FLAN model is not loaded.")

        with self._lock:
            inputs = self.tokenizer(
                prompt,
                return_tensors="pt",
                truncation=True,
                max_length=1024,
            )

            outputs = self.model.generate(
                **inputs,
                max_new_tokens=220,
                temperature=0.2,
                do_sample=True,
            )

            text = self.tokenizer.decode(outputs[0], skip_special_tokens=True)
            return text.strip()

    def _is_bad_rewrite(self, text: str) -> bool:
        if not text:
            return True

        stripped = text.strip().lower()

        # Too short or generic
        if len(stripped) < 80:
            return True

        # Common weak patterns from small/medium models
        bad_patterns = [
```

```
            "the doctor will",
            "a doctor may be able",
            "provide a brief",
            "clear and reassuring answer",
            "the first step",
        ]

        if any(p in stripped for p in bad_patterns):
            return True

        return False

    def explain_model_only(self, payload: Dict[str, Any]) -> str:
        """
        Debug helper: returns only the raw FLAN rewrite result.
        """
        if not self.model_loaded:
            return f"MODEL NOT LOADED: {self.load_error}"

        structured_text = self._build_structured_explanation(payload)
        prompt = self._build_rewrite_prompt(structured_text)
        return self._generate_with_flan(prompt)

    def explain(self, payload: Dict[str, Any], use_ai: bool = False) -> str:
        """
        Default:
        - use_ai=False -> deterministic explanation
        - use_ai=True  -> try FLAN rewrite, else fallback safely
        """
        structured_text = self._build_structured_explanation(payload)

        if not use_ai:
            return structured_text

        if not self.model_loaded:
            return structured_text

        try:
            rewritten = self.explain_model_only(payload)
            if self._is_bad_rewrite(rewritten):
                return structured_text
            return rewritten
        except Exception:
            return structured_text
```

Test Deterministic Explanation:

Run in terminal:

python -c "from core.explainer import AIExplainer; ex=AIExplainer(); payload={'top_diagnosis': {'diagnosis':'Possible angina or heart-related issue','specialty':'Cardiology','triage_level':'emergency','recommended_tests':'ECG, troponin blood test, chest X-ray','advice':'Seek emergency care immediately.'}, 'doctors':[{'name':'Dr. Park Jihoon','specialty':'Cardiology','city':'Seoul','rating':4.7}], 'insurance_text':'Estimated insurance cost: 5,407', 'history_summary':'No previous visit found.'}; print(ex.explain(payload, use_ai=False))"

Output Expected:

```
python -c "from core.explainer import AIExplainer; ex=AIExplainer(); payload={'top_diagnosis': {'diagnosis
':'Possible angina or heart-related issue','specialty':'Cardiology','triage_level':'emergency','recommended_tests':'ECG, troponin blood test, chest X-ray','ad
vice':'Seek emergency care immediately.'}, 'doctors':[{'name':'Dr. Park Jihoon','specialty':'Cardiology','city':'Seoul','rating':4.7}], 'insurance_text':'Esti
mated insurance cost: 5,407', 'history_summary':'No previous visit found.'}; print(ex.explain(payload, use_ai=False))"
Here is a simple explanation of the result.

Based on the symptoms entered, the system found a health insight that is most closely related to: Possible angina or heart-related issue.
The recommended next step is to consult a Cardiology specialist.
The urgency level for this case is marked as emergency.
Suggested tests include: ECG, troponin blood test, chest X-ray.
General advice from the support system is: Seek emergency care immediately.

Estimated insurance cost: 5,407

History note: No previous visit found.

Suggested doctors based on the selected city and specialty include:
- Dr. Park Jihoon in Seoul (Cardiology, rating 4.7)

This explanation is for educational decision-support purposes only and is not a medical diagnosis. Please consult a licensed healthcare professional.
```

Test FLAN-Large raw rewrite:

python -c "from core.explainer import AIExplainer; ex=AIExplainer(use_local_model=True); payload={'top_diagnosis': {'diagnosis':'Possible angina or heart-related issue','specialty':'Cardiology','triage_level':'emergency','recommended_tests':'ECG, troponin blood test, chest X-ray','advice':'Seek emergency care immediately.'}, 'doctors':[{'name':'Dr. Park Jihoon','specialty':'Cardiology','city':'Seoul','rating':4.7}], 'insurance_text':'Estimated insurance cost: 5,407', 'history_summary':'No previous visit found.'}; print(ex.explain_model_only(payload))"

Quanskill

Output Expected:

```
(.venv) PS D:\healthcare ai agent\healthcare_agent> python -c "from core.explainer import AIExplainer; ex=AIExplainer(use_local_model=True); payload={'top_dia
gnosis': {'diagnosis':'Possible angina or heart-related issue','specialty':'Cardiology','triage_level':'emergency','recommended_tests':'ECG, troponin blood te
st, chest X-ray','advice':'Seek emergency care immediately.'}, 'doctors':[{'name':'Dr. Park Jihoon','specialty':'Cardiology','city':'Seoul','rating':4.7}], 'i
nsurance_text':'Estimated insurance cost: 5,407', 'history_summary':'No previous visit found.'}; print(ex.explain_model_only(payload))"
Loading explainer model: google/flan-t5-large
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100%|                                                                                                    | 662/662 [00:00<00:00, 1.84MB/s]
D:\healthcare ai agent\healthcare_agent\.venv\Lib\site-packages\huggingface_hub\file_download.py:129: UserWarning: `huggingface_hub` cache-system uses symlink
s by default to efficiently store duplicated files but your machine does not support them in C:\Users\simba\.cache\huggingface\hub\models--google--flan-t5-lar
ge. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DI
SABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this
 article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
  warnings.warn(message)
tokenizer_config.json: 2.54kB [00:00, 2.56MB/s]
spiece.model: 100%|                                                                                                   | 792k/792k [00:02<00:00, 298kB/s]
tokenizer.json: 2.42MB [00:00, 51.3MB/s]
special_tokens_map.json: 2.20kB [00:00, 4.36MB/s]
model.safetensors: 100%|                                                                                              | 3.13G/3.13G [00:45<00:00, 68.4MB/s]
Loading weights: 100%|                                                                                                | 558/558 [00:00<00:00, 2268.82it/s]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT t
ie them. You should update the config with `tie_word_embeddings=False` to silence this warning
generation_config.json: 100%|                                                                                         | 147/147 [00:00<00:00, 446kB/s]
Explainer model loaded successfully.
Based on the symptoms entered, the support system found a health insight that is most closely related to: Possible angina or heart-related issue. The recommen
ded next step is to consult a Cardiology specialist. The urgency level for this case is marked as emergency. Suggested tests include: ECG, troponin blood test
, chest X-ray. General advice from the support system is: Seek emergency care immediately. Estimated insurance cost: 5,407 History note: No previous visit fou
nd. Suggested doctors based on the selected city and specialty include: - Dr. Park Jihoon in Seoul (Cardiology, rating 4.7) This explanation is for educationa
l decision-support purposes only and is not a medical diagnosis. Please consult a licensed healthcare professional.
```

Test final smart behavior:

python -c "from core.explainer import AIExplainer; ex=AIExplainer(use_local_model=True); payload={'top_diagnosis': {'diagnosis':'Possible angina or heart-related issue','specialty':'Cardiology','triage_level':'emergency','recommended_tests':'ECG, troponin blood test, chest X-ray','advice':'Seek emergency care immediately.'}, 'doctors':[{'name':'Dr. Park Jihoon','specialty':'Cardiology','city':'Seoul','rating':4.7}], 'insurance_text':'Estimated insurance cost: 5,407', 'history_summary':'No previous visit found.'}; print(ex.explain(payload, use_ai=True))"

Output Expected:

```
(.venv) PS D:\healthcare ai agent\healthcare_agent> python -c "from core.explainer import AIExplainer; ex=AIExplainer(use_local_model=True); payload={'top_dia
gnosis': {'diagnosis':'Possible angina or heart-related issue','specialty':'Cardiology','triage_level':'emergency','recommended_tests':'ECG, troponin blood te
st, chest X-ray','advice':'Seek emergency care immediately.'}, 'doctors':[{'name':'Dr. Park Jihoon','specialty':'Cardiology','city':'Seoul','rating':4.7}], 'i
nsurance_text':'Estimated insurance cost: 5,407', 'history_summary':'No previous visit found.'}; print(ex.explain(payload, use_ai=True))"
Loading explainer model: google/flan-t5-large
Loading weights: 100%|                                                                                                | 558/558 [00:00<00:00, 1749.36it/s]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT t
ie them. You should update the config with `tie_word_embeddings=False` to silence this warning
Explainer model loaded successfully.
Based on the symptoms entered, the support system found a health insight that is most closely related to: Possible angina or heart-related issue. The recommen
ded next step is to consult a Cardiology specialist. The urgency level for this case is marked as emergency. Suggested tests include: ECG, troponin blood test
, chest X-ray. General advice from the support system is: Seek emergency care immediately. Estimated insurance cost: 5,407 History note: No previous visit fou
nd. Suggested doctors based on the selected city and specialty include: - Dr. Park Jihoon in Seoul (Cardiology, rating 4.7) This explanation is for educationa
l decision-support purposes only and is not a medical diagnosis. Please consult a licensed healthcare professional.
```

Step 4: Time to build User Interface

Make sure `ui` package exists

Run:

```
ni ui\__init__.py
```

If it says the file already exists, that is fine.

Replace ui\app.py with the final fixed version

Create/open (Run in Terminal):

ni ui\app.py

Then paste this entire code into `ui\app.py` in the VS Code editor.

```
import gradio as gr

from core.diagnostics import SymptomDiagnosticEngine
from core.doctors import find_doctors, create_doctor_map
from core.insurance import InsuranceEstimator
from core.memory import generate_patient_id, update_patient_memory,
get_patient_history
from core.explainer import AIExplainer


diagnostic_engine = SymptomDiagnosticEngine()
insurance_estimator = InsuranceEstimator()
ai_explainer = AIExplainer()  # fallback by default


def calculate_bmi(height_cm, weight_kg):
    height_m = float(height_cm) / 100.0
    if height_m <= 0:
        raise ValueError("Height must be greater than zero.")
    bmi = float(weight_kg) / (height_m ** 2)
    return round(bmi, 2)


def format_doctors(doctors):
    if not doctors:
        return "No doctors found."
```

```
    lines = []
    for doc in doctors:
        lines.append(
            f"{doc['name']} | {doc['specialty']} | {doc['city']} | Rating:
{doc['rating']}"
        )
    return "\n".join(lines)


def format_history(history):
    if not history:
        return "No previous patient history found."

    visits = history.get("history", [])
    if not visits:
        return "No previous patient history found."

    lines = []
    lines.append("Patient Visit Timeline")
    lines.append("-" * 40)

    for idx, visit in enumerate(visits, start=1):
        diag = visit.get("diagnosis", {})
        top = diag.get("top_diagnosis", {}) if isinstance(diag, dict) else {}
        demographics = history.get("demographics", {})

        lines.append(f"Visit {idx}")
        lines.append(f"Timestamp: {visit.get('timestamp', 'Unknown')}")
        lines.append(f"Symptoms: {visit.get('symptoms', 'Unknown')}")
        lines.append(f"Health insight: {top.get('diagnosis', 'Unknown')}")
        lines.append(f"Recommended specialty: {top.get('specialty', 'Unknown')}")
        lines.append(f"Insurance estimate: {visit.get('insurance_price',
'Unknown')}")
        if demographics.get("bmi") is not None:
            lines.append(f"Stored BMI: {demographics.get('bmi')}")
        lines.append("-" * 40)

    return "\n".join(lines)


def format_summary(result):
    top = result.get("top_diagnosis")
    if not top:
        return "No health insight found."
```

```
    summary_lines = [
        f"Health Insight: {top.get('diagnosis', 'Unknown')}",
        f"Recommended Specialty: {top.get('specialty', 'Unknown')}",
        f"Urgency Level: {top.get('triage_level', 'Unknown')}",
    ]

    if result.get("calculated_bmi") is not None:
        summary_lines.append(f"Calculated BMI: {result.get('calculated_bmi')}")

    return "\n".join(summary_lines)


def format_detailed_analysis(result):
    top = result.get("top_diagnosis")
    if not top:
        return "No detailed analysis found."

    lines = []
    lines.append("Structured Health Analysis")
    lines.append("-" * 40)
    lines.append(f"Matched symptom pattern: {top.get('symptom_pattern',
'Unknown')}")
    lines.append(f"Health insight: {top.get('diagnosis', 'Unknown')}")
    lines.append(f"Recommended specialty: {top.get('specialty', 'Unknown')}")
    lines.append(f"Urgency level: {top.get('triage_level', 'Unknown')}")
    lines.append(f"Suggested tests: {top.get('recommended_tests', 'Unknown')}")
    lines.append(f"Advice: {top.get('advice', 'Unknown')}")
    lines.append(f"Similarity confidence: {top.get('confidence', 0):.4f}")

    if result.get("calculated_bmi") is not None:
        lines.append(f"Calculated BMI: {result.get('calculated_bmi')}")

    if result.get("history_summary"):
        lines.append("")
        lines.append(f"History summary: {result['history_summary']}")

    if result.get("safety_disclaimer"):
        lines.append("")
        lines.append(result["safety_disclaimer"])

    return "\n".join(lines)


def analyze(name, email, age, sex, height_cm, weight_kg, children, smoker,
region, symptoms, city, use_ai):
```

```
    try:
        if not name.strip() or not email.strip() or not symptoms.strip():
            msg = "Please fill in name, email, and symptoms."
            return msg, msg, msg, msg, msg, msg, "<p>No map available.</p>"

        patient_id = generate_patient_id(name, email)

        result = diagnostic_engine.diagnose(symptoms, patient_id)
        top = result.get("top_diagnosis")

        if not top:
            msg = "No health insight could be generated."
            return msg, msg, msg, msg, msg, msg, "<p>No map available.</p>"

        doctors = find_doctors(top.get("specialty", ""), city)

        bmi = calculate_bmi(height_cm, weight_kg)
        result["calculated_bmi"] = bmi

        insurance_price, insurance_text = insurance_estimator.estimate(
            patient_id=patient_id,
            age=int(age),
            sex=sex,
            bmi=float(bmi),
            children=int(children),
            smoker=smoker,
            region=region,
        )

        if use_ai and not ai_explainer.model_loaded:
            ai_explainer.enable_local_model()

        payload = {
            "top_diagnosis": top,
            "doctors": doctors,
            "insurance_text": insurance_text,
            "history_summary": result.get("history_summary", ""),
        }

        ai_text = ai_explainer.explain(payload, use_ai=use_ai)

        demographics = {
            "name": name,
            "email": email,
            "age": age,
```

```
            "sex": sex,
            "height_cm": height_cm,
            "weight_kg": weight_kg,
            "bmi": bmi,
            "children": children,
            "smoker": smoker,
            "region": region,
            "city": city,
        }

        update_patient_memory(
            patient_id=patient_id,
            demographics=demographics,
            symptoms=symptoms,
            diagnosis_result=result,
            insurance_price=insurance_price,
            doctors=doctors,
        )

        history = get_patient_history(patient_id)

        detailed_analysis = format_detailed_analysis(result)
        summary = format_summary(result)
        doctor_text = format_doctors(doctors)
        history_text = format_history(history)
        doctor_map_html = create_doctor_map(doctors)

        return (
            detailed_analysis,
            insurance_text,
            summary,
            ai_text,
            history_text,
            doctor_text,
            doctor_map_html,
        )

    except Exception as e:
        msg = f"Error during analysis: {e}"
        return msg, msg, msg, msg, msg, msg, "<p>No map available.</p>"


def show_history(name, email):
    try:
        if not name.strip() or not email.strip():
```

```
            return "Please enter both name and email."
        patient_id = generate_patient_id(name, email)
        history = get_patient_history(patient_id)
        return format_history(history)
    except Exception as e:
        return f"Error while loading history: {e}"


with gr.Blocks(title="Healthcare AI Agent") as demo:
    gr.Markdown("# Healthcare AI Agent")
    gr.Markdown("From Data → Intelligence → AI Product")

    with gr.Row():
        with gr.Column():
            name = gr.Textbox(label="Full Name")
            email = gr.Textbox(label="Email")
            age = gr.Number(label="Age", value=35)
            sex = gr.Dropdown(
                choices=["male", "female"],
                value="male",
                label="Sex",
            )
            height_cm = gr.Number(label="Height (cm)", value=170)
            weight_kg = gr.Number(label="Weight (kg)", value=70)
            children = gr.Number(label="Children", value=1)
            smoker = gr.Dropdown(
                choices=["yes", "no"],
                value="no",
                label="Smoker",
            )
            region = gr.Dropdown(
                choices=["northeast", "northwest", "southeast", "southwest"],
                value="southeast",
                label="Region",
            )
            city = gr.Dropdown(
                choices=["Seoul", "Hanoi", "Ho Chi Minh City", "Da Nang"],
                value="Seoul",
                label="City",
            )
            symptoms = gr.Textbox(
                label="Describe Symptoms",
                lines=4,
                placeholder="Example: chest pain and sweating",
            )
```

```
            use_ai = gr.Checkbox(
                label="Use local AI explanation (experimental)",
                value=False,
            )

            analyze_button = gr.Button("Analyze")
            history_button = gr.Button("Show patient history")

        with gr.Column():
            detailed_analysis = gr.Textbox(
                label="Full analysis",
                lines=12,
            )
            insurance_box = gr.Textbox(
                label="Insurance estimate",
                lines=2,
            )
            summary_box = gr.Textbox(
                label="Health insight summary",
                lines=4,
            )
            ai_box = gr.Textbox(
                label="Explanation layer output",
                lines=8,
            )
            history_box = gr.Textbox(
                label="Patient history",
                lines=10,
            )
            doctors_box = gr.Textbox(
                label="Doctor recommendations",
                lines=6,
            )
            doctor_map = gr.HTML(label="Doctor map")

    analyze_button.click(
        analyze,
        inputs=[name, email, age, sex, height_cm, weight_kg, children, smoker,
region, symptoms, city, use_ai],
        outputs=[
            detailed_analysis,
            insurance_box,
            summary_box,
            ai_box,
            history_box,
```

```
            doctors_box,
            doctor_map,
        ],
    )

    history_button.click(
        show_history,
        inputs=[name, email],
        outputs=history_box,
    )


if __name__ == "__main__":
    demo.launch()
```

Update your core\memory.py by replacing ur current core\memory.py code with below:

```
import json
import os
import hashlib
from datetime import datetime, timezone
from typing import Dict, Any, Optional, List

from .config import PATIENT_MEMORY_PATH, STORAGE_DIR


def generate_patient_id(name: str, email: str) -> str:
    key = f"{name.strip().lower()}|{email.strip().lower()}"
    return hashlib.sha256(key.encode("utf-8")).hexdigest()[:16]


def load_memory() -> Dict[str, Any]:
    os.makedirs(STORAGE_DIR, exist_ok=True)
    if not os.path.exists(PATIENT_MEMORY_PATH):
        return {}
    try:
        with open(PATIENT_MEMORY_PATH, "r", encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError:
        return {}


def save_memory(data: Dict[str, Any]) -> None:
    os.makedirs(STORAGE_DIR, exist_ok=True)
    with open(PATIENT_MEMORY_PATH, "w", encoding="utf-8") as f:
```

```python
        json.dump(data, f, ensure_ascii=False, indent=2)


def get_patient_history(patient_id: str) -> Optional[Dict[str, Any]]:
    memory = load_memory()
    return memory.get(patient_id)


def _parse_timestamp(ts: str) -> Optional[datetime]:
    try:
        return datetime.fromisoformat(ts.replace("Z", "+00:00"))
    except Exception:
        return None


def _is_same_recent_visit(last_entry: Dict[str, Any], new_symptoms: str,
window_minutes: int = 5) -> bool:
    last_symptoms = str(last_entry.get("symptoms", "")).strip().lower()
    current_symptoms = str(new_symptoms).strip().lower()

    if last_symptoms != current_symptoms:
        return False

    last_timestamp = _parse_timestamp(last_entry.get("timestamp", ""))
    if last_timestamp is None:
        return False

    now = datetime.now(timezone.utc)
    delta_minutes = (now - last_timestamp).total_seconds() / 60.0

    return delta_minutes <= window_minutes


def update_patient_memory(
    patient_id: str,
    demographics: Dict[str, Any],
    symptoms: str,
    diagnosis_result: Dict[str, Any],
    insurance_price: Optional[float],
    doctors: List[Dict[str, Any]],
) -> None:
    memory = load_memory()

    if patient_id not in memory:
        memory[patient_id] = {
            "demographics": demographics,
```

```
            "history": []
        }

    # always refresh demographics to latest values
    memory[patient_id]["demographics"] = demographics

    new_entry = {
        "timestamp": datetime.now(timezone.utc).isoformat().replace("+00:00",
"Z"),
        "symptoms": symptoms,
        "diagnosis": diagnosis_result,
        "insurance_price": insurance_price,
        "doctors": doctors,
    }

    history = memory[patient_id]["history"]

    # If the latest visit is actually the same recent visit, overwrite it
    if history:
        last_entry = history[-1]
        if _is_same_recent_visit(last_entry, symptoms, window_minutes=5):
            history[-1] = new_entry
            save_memory(memory)
            return

    # Otherwise append as a genuinely new visit
    history.append(new_entry)
    save_memory(memory)
```

Step 5: Run the app

In terminal, from the project root:

```
python -m ui.app
```

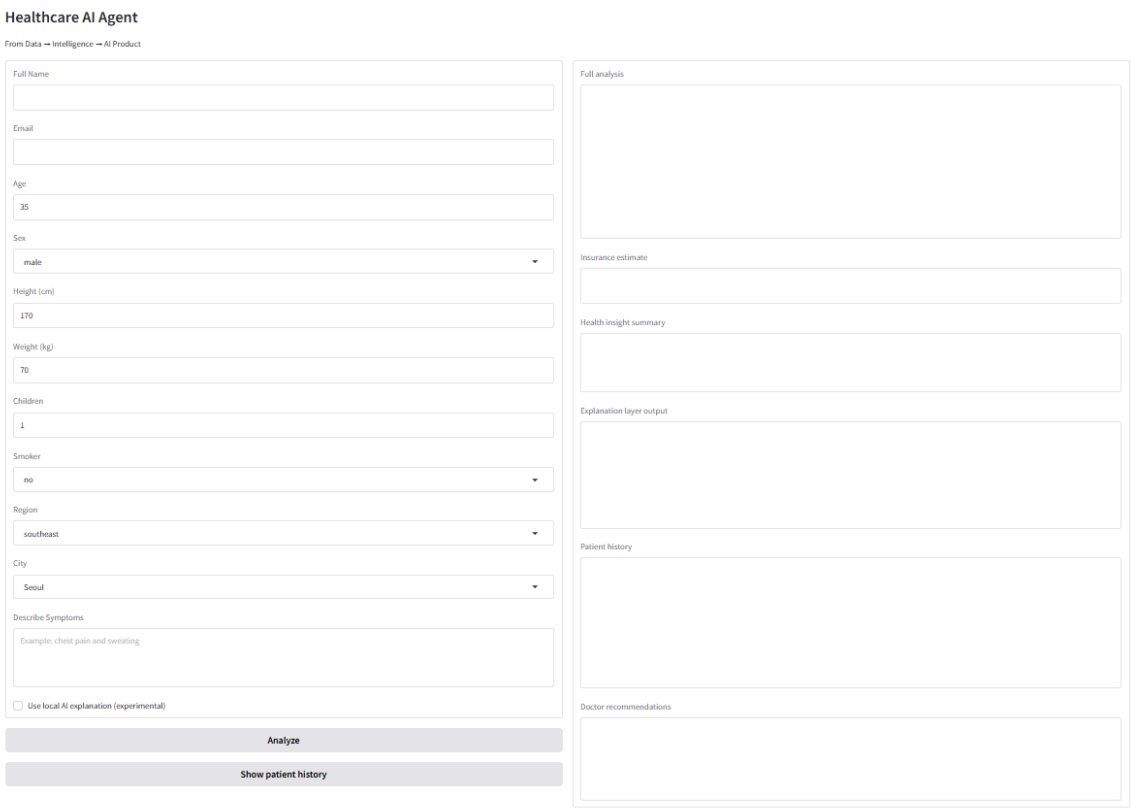
Expected output should show something like:

```
Running on local URL:  http://127.0.0.1:7860
```

Open that link in your browser.

Expected Look:

```
(.venv) PS D:\healthcare ai agent\healthcare_agent> python -m ui.app
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|____________________________________________________________| 103/103 [00:00<00:00, 1916.50it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.
Loading explainer model: google/flan-t5-large
Loading weights: 100%|____________________________________________________________| 558/558 [00:00<00:00, 1152.42it/s]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the checkpoints, so we will NOT t
ie them. You should update the config with `tie_word_embeddings=False` to silence this warning
Explainer model loaded successfully.
```

Expected UI: